

Lab 3 – Raft-Lite Leader Election and Consensus

Omirkul Rauan
Distributed Systems Course

January 23, 2026

1 Objectives

The goal of this lab was to implement a simplified version of the Raft consensus algorithm, focusing only on leader election and heartbeats. We then deployed 3 nodes on AWS EC2 and observed election and failover behavior.

This report covers:

- System model and assumptions
- Design & implementation of election and heartbeats
- Experimental results (normal start, leader crash)
- Conclusions

2 System Model

- **Roles:** Follower, Candidate, Leader
- **Communication:** HTTP + JSON via endpoints `/status`, `/vote`, `/heartbeat`
- **Cluster:** 3 nodes on EC2 t3.micro (same region)
- **Election timeout:** random 150–300 ms
- **Heartbeat interval:** 50 ms
- **Majority:** $> N/2$ votes (2 out of 3)

3 Implementation Highlights

Main file: `raft_node.py`

Key state variables (protected by lock):

- `current_term` (int)
- `voted_for` (str or None)
- `role` (enum: Follower / Candidate / Leader)
- `current_leader` (str or None)
- `last_heartbeat` (float timestamp)

3.1 Vote Request Handler (simplified)

```
def handle_vote_request(term: int, candidate_id: str) -> dict:
    global current_term, voted_for, role, last_heartbeat, current_leader
    with lock:
        vote_granted = False
        if term < current_term:
            pass # stale, reject
        else:
            if term > current_term:
                current_term = term
                voted_for = None
                role = Role.FOLLOWER
                current_leader = None
            if voted_for is None or voted_for == candidate_id:
                voted_for = candidate_id
                vote_granted = True
                last_heartbeat = time.time()
    return {"term": current_term, "vote_granted": vote_granted}
```

Heartbeats follow similar logic: update term if higher, refresh timeout, set leader.
Election & heartbeat loops run in background threads.

4 Experiments

4.1 Scenario A – Normal startup

All 3 nodes started within 3 seconds.

Result:

Leader elected: Node B

Term: 1

Reason: Node B had shortest timeout (172 ms), became candidate first, collected 2 votes quickly.

4.2 Scenario B – Leader crash

Leader (Node B) killed with Ctrl+C.

Result:

New leader: Node C

Failover time: 0.41 seconds

New term: 2

Explanation: 210 ms until first timeout, then 200 ms for vote round-trips.

5 Discussion & Conclusion

Elections were stable (no split votes in 7+ restarts). Failover consistently 300–550 ms — matches timeout range + network delay. Randomized timeouts + short heartbeats worked as intended.

Future steps: log replication, persistence, partition handling.